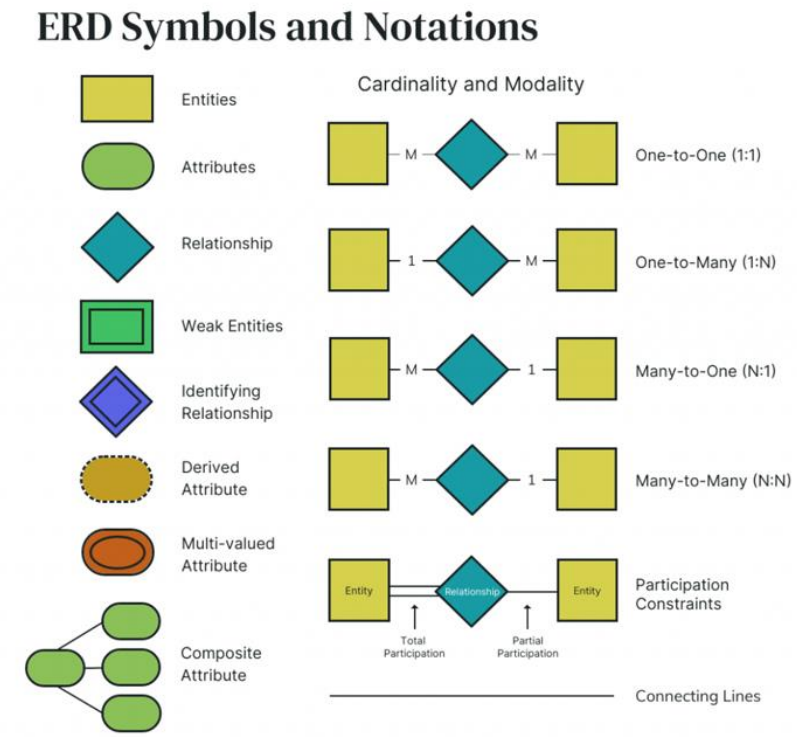
SQL Reference

Command	Description	Syntax	Example
CREATE / ALTER TABLE	Creates a table, or adds a column to an existing table	CREATE TABLE t (col type, ...); ALTER TABLE t ADD COLUMN col type;	CREATE TABLE s (id SERIAL PRIMARY KEY, name VARCHAR(50) NOT NULL);
Data types	INT; SERIAL (auto-incrementing integer); NUMERIC(p,s) exact decimal; VARCHAR(n); TEXT; DATE; BOOLEAN	col_name type	price NUMERIC(8,2)
Constraints	PRIMARY KEY (also composite), NOT NULL, DEFAULT, FOREIGN KEY	col type REFERENCES t(col) PRIMARY KEY (a, b) col type DEFAULT value	dept_id INT REFERENCES dept(dept_id); PRIMARY KEY (s_id, c_id)
INSERT / UPDATE / DELETE	Adds rows, changes values, removes rows (always use WHERE with UPDATE and DELETE)	INSERT INTO t (c1, c2) VALUES (v1, v2); UPDATE t SET c = expr WHERE cond; DELETE FROM t WHERE cond;	UPDATE s SET age = age + 1 WHERE id = 3;
SELECT / DISTINCT / AS / LIMIT	Chooses columns, removes duplicate rows, renames an output column, limits the number of rows	SELECT DISTINCT c AS alias FROM t LIMIT n;	SELECT DISTINCT age AS a FROM s LIMIT 5;
WHERE operators	Filters rows: =, <>, <, >, AND, OR, NOT, IS NULL, BETWEEN (inclusive), IN, LIKE (% = any text, _ = one character)	WHERE c BETWEEN a AND b WHERE c IN (v1, v2) WHERE c LIKE 'A%'	WHERE age BETWEEN 20 AND 25 AND email IS NOT NULL
ORDER BY	Sorts the result; ASC is the default; NULLS FIRST / NULLS LAST places NULL values	ORDER BY c1 DESC, c2 ASC	ORDER BY age DESC, name
INNER JOIN / LEFT JOIN	INNER keeps matching rows only; LEFT keeps every row of the left table (NULL when there is no match)	FROM a JOIN b ON a.k = b.k FROM a LEFT JOIN b ON a.k = b.k	FROM s LEFT JOIN c ON s.id = c.s_id WHERE c.s_id IS NULL
Self join	Joins a table to itself using two aliases	FROM t x JOIN t y ON x.fk = y.pk	FROM s a JOIN s b ON a.mentor_id = b.id
Subquery / EXISTS	A query inside parentheses; EXISTS is true when the subquery returns at least one row	WHERE c > (SELECT AVG(c) FROM t) WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.k = t1.k)	WHERE age > (SELECT AVG(age) FROM s)
WITH (CTE)	A temporary named result that can be used inside one query	WITH name AS (SELECT ...) SELECT ... FROM name;	WITH a AS (SELECT dept, AVG(age) AS av FROM s GROUP BY dept) SELECT * FROM a;
UNION / INTERSECT / EXCEPT	Combines two queries that have the same columns: all distinct rows / rows in both / rows of the first that are not in the second	q1 UNION q2 q1 INTERSECT q2 q1 EXCEPT q2	SELECT name FROM s EXCEPT SELECT name FROM t

Command	Description	Syntax	Example
COUNT / SUM / AVG / MIN / MAX / ROUND	Aggregate functions (they ignore NULL, COUNT(*) counts rows); ROUND(x, n) rounds to n decimals	COUNT(*), AVG(c), ROUND(AVG(c), 1)	SELECT ROUND(AVG(age), 1) FROM s;
GROUP BY / HAVING	Groups rows for aggregates; WHERE filters rows before grouping, HAVING filters groups after it	SELECT c, COUNT(*) FROM t WHERE cond GROUP BY c HAVING COUNT(*) > 1	SELECT dept, COUNT(*) FROM s GROUP BY dept HAVING COUNT(*) > 2;
OVER / PARTITION BY	Window function: calculates across related rows without collapsing them	func() OVER (PARTITION BY c1 ORDER BY c2)	AVG(age) OVER (PARTITION BY dept)
ROW_NUMBER / RANK / DENSE_RANK	Unique number / rank with gaps after ties (1, 1, 3) / rank without gaps (1, 1, 2)	RANK() OVER (ORDER BY c DESC)	ROW_NUMBER() OVER (ORDER BY age DESC)

ER Notation



Question 1: University Database (50 points)

Use PostgreSQL syntax. The database has three tables (PK = primary key, FK = foreign key):

```
faculties(faculty_id PK, faculty_name, city)
students(student_id PK, name, age, email, scholarship, faculty_id FK, mentor_id FK, enroll_date)
tutors(tutor_id PK, name, hourly_rate)
```

students.faculty_id refers to faculties.faculty_id, and students.mentor_id refers to students.student_id.

The tables contain this data:

```
INSERT INTO faculties VALUES
  (10, 'Engineering', 'Baku'), (20, 'Medicine', 'Baku'),
  (30, 'Law', 'Ganja'), (40, 'Economics', 'Sumgait'),
  (50, 'Arts', 'Ganja');

INSERT INTO students VALUES
  (1, 'Aynur',   28, 'aynur@uni.az',   500, 20, NULL, '2019-09-01'),
  (2, 'Rashad',  25, 'rashad@uni.az',  350, 20, 1,    '2021-09-01'),
  (3, 'Lala',    22, NULL,             300, 20, 2,    '2022-09-05'),
  (4, 'Orkhan',  24, 'orkhan@uni.az',  280, 10, 1,    '2020-09-01'),
  (5, 'Nurlan',  21, 'nurlan@uni.az',  280, 10, 4,    '2023-09-04'),
  (6, 'Vugar',   27, NULL,             420, 30, 1,    '2018-09-03'),
  (7, 'Sevinj',  19, 'sevinj@uni.az',  220, 30, 6,    '2024-09-02'),
  (8, 'Kanan',   29, 'kanan@uni.az',   600, 40, 1,    '2017-09-01'),
  (9, 'Ismayil', 23, 'ismayil@uni.az', 350, 40, 8,    '2022-02-10'),
  (10,'Altay',   26, 'altay@uni.az',   300, NULL, 1,  '2021-02-08');

INSERT INTO tutors VALUES
  (1, 'Sevinj', 25.00), (2, 'Elnur', 30.00),
  (3, 'Ismayil', 28.50), (4, 'Tural', 22.00);
```

1. Draw ER diagram. Use Entity, Attributes, Relationships and Cardinality.

2. Create and change data (DDL and DML). Write the statements for:
    a. a table `courses` with `course_id` (auto-incrementing primary key), `title` (text up to 60 characters, required), `tuition` (number with 2 decimals, up to 10 digits), `start_date` (date) and `faculty_id` (foreign key to `faculties`);
    b. adding a column `status` (up to 20 characters, default `'open'`) to `courses`;
3. Select, filter and sort.
    a. Return `student_name` (the name), `scholarship` and `enroll_date` of students who study in faculty 10, 20, 30 or 40, receive a scholarship between 250 and 500 (both included), have an email (not NULL) and whose name does not start with A. Show the highest scholarship first, equal scholarships by name from A to Z, and only the first 3 rows.
    b. Return each city where a faculty is located once, in alphabetical order.
4. Joins.
    a. Return `name`, `faculty_name` and `city` of every student that has a faculty, ordered by name.
    b. Return the `faculty_name` of every faculty that has no students (use a join and IS NULL, not a subquery).
5. Subqueries, CTE and set operations.
    a. Return `name` and `scholarship` of students who receive more than the average scholarship of all students (do not type the average), highest scholarship first.

b. Using EXISTS, return the `faculty_name` of faculties that have at least one student receiving more than 400, in alphabetical order.

6. Aggregation and window functions.

   (a) For students older than 22, return `faculty_id`, `total_students` and `avg_scholarship` (rounded to 1 decimal) for each faculty, but only for faculties with at least 2 such students; highest average first.

   (b) Return `name`, `scholarship`, `row_num`, `rnk` and `dense_rnk` for all students, ranked university-wide by scholarship from highest to lowest (`row_num` breaks ties by name); order by scholarship DESC, name.